

Aggregate Functions in SQL

Introduction to Databases

MIN() and MAX() Functions

- The MIN() function returns the smallest value of the selected column.
- The MAX() function returns the largest value of the selected column.
- SELECT **MIN**(*column_name*)
FROM *table_name*
WHERE *condition*;
- SELECT **MAX**(*column_name*)
FROM *table_name*
WHERE *condition*;

Example

- `SELECT MIN(Price) AS SmallestPrice
FROM Products;`
- `SELECT MAX(Price) AS LargestPrice
FROM Products;`

MySQL COUNT()

- The COUNT() function returns the number of rows that matches a specified criterion.
- ```
SELECT COUNT(column_name)
FROM table_name
WHERE condition;
```
- ```
SELECT COUNT(ProductID)  
FROM Products;
```

Count(*)

- COUNT(*) returns the number of rows in a specified table
- It preserves duplicate rows. It counts each row separately.
- This includes rows that contain null values.
- COUNT(*) without GROUP BY returns the cardinality (number of rows) in the result set. This includes rows comprised of all-NULL values and duplicates.
- COUNT(*) with GROUP BY returns the number of rows in each group. This includes NULL values and duplicates.

Count(*)

- COUNT(ALL <expression>) evaluates expression for each row in a group, and returns the number of non null values.
- COUNT(DISTINCT *expression*) evaluates expression for each row in a group, and returns the number of unique, non null values.

Example

- `SELECT COUNT(*) FROM HumanResources.Employee;`

- Output:

290

- `SELECT COUNT(DISTINCT Title) FROM HumanResources.Employee;`

- Output:

67

MySQL AVG()

- The AVG() function returns the average value of a numeric column.
- SQL **AVG** function is used to find out the average of a field in various records.
- This function returns the average of the values in a group. It ignores null values.

- Syntax

```
SELECT AVG(column_name)  
FROM table_name  
WHERE condition;
```


Example

- `SELECT AVG(Price)
FROM Products;`

MySQL SUM()

- The SUM() function returns the total sum of a numeric column.
- Returns the sum of all the values, or only the DISTINCT values, in the expression.
- SUM can be used with numeric columns only.
- Null values are ignored.
- Syntax

```
SELECT SUM(column_name)  
FROM table_name  
WHERE condition;
```

Example

- `SELECT SUM(Quantity)
FROM OrderDetails;`
- OR
- `SELECT SUM(DISTINCT Quantity)
FROM OrderDetails;`

MySQL IFNULL()

- The MySQL IFNULL() function lets you return an alternative value if an expression is NULL.
- The IFNULL() function returns a specified value if the expression is NULL.
- If the expression is NOT NULL, this function returns the expression.

MySQL IFNULL()

- Syntax

IFNULL(*expression*, *alt_value*)

<i>expression</i>	Required. The expression to test whether is NULL
<i>alt_value</i>	Required. The value to return if <i>expression</i> is NULL

Example:

```
SELECT ProductName, UnitPrice * (UnitsInStock +  
IFNULL(UnitsOnOrder, 0))  
FROM Products;
```

ORDER BY Clause

- The ORDER BY clause can be used in the SELECT query to sort the result in ascending or descending order of one or more columns.
- The ORDER BY keyword is used to sort the result-set in ascending or descending order.
- The ORDER BY keyword sorts the records in ascending order by default.
- To sort the records in descending order, use the DESC keyword.

Order By Syntax | Example

- `SELECT column1, column2, ...`
`FROM table_name`
`ORDER BY column1, column2, ... ASC|DESC;`
- `SELECT * FROM Customers`
`ORDER BY Country;`
- `SELECT * FROM Customers`
`ORDER BY Country DESC;`

MySQL GROUP BY Statement

- The GROUP BY statement groups rows that have the same values into summary rows, like "find the number of customers in each country".
- The SQL **GROUP BY** clause is used in collaboration with the SELECT statement to arrange identical data into groups.
- The GROUP BY statement is often used with aggregate functions (COUNT(), MAX(), MIN(), SUM(), AVG()) to group the result-set by one or more columns.

GROUP BY Syntax

- SELECT *column_name(s)*
FROM *table_name*
WHERE *condition*
GROUP BY *column_name(s)*
ORDER BY *column_name(s)*;

Example

- `SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country;`
- `SELECT DeptId, COUNT(EmplId) as 'Number of Employees' FROM
Employee GROUP BY DeptId;`
- `SELECT dept.Name, sum(emp.salary) as 'Total Salaries' FROM
Employee emp, Department dept WHERE emp.deptid = dept.DeptId
GROUP by dept.Name`

- The HAVING clause was added to SQL because the WHERE keyword cannot be used with aggregate functions.
- The **HAVING Clause** enables you to specify conditions that filter which group results appear in the results.
- The WHERE clause places conditions on the selected columns, whereas the HAVING clause places conditions on groups created by the GROUP BY clause.

HAVING Syntax

- SELECT *column_name(s)*
FROM *table_name*
WHERE *condition*
GROUP BY *column_name(s)*
HAVING *condition*
ORDER BY *column_name(s)*;

Example

- `SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5;`
- `SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5
ORDER BY COUNT(CustomerID) DESC;`

Example

```
SELECT Employees.LastName, COUNT(Orders.OrderID) AS NumberOfOrders  
FROM (Orders  
INNER JOIN Employees ON Orders.EmployeeID =  
Employees.EmployeeID)  
GROUP BY LastName  
HAVING COUNT(Orders.OrderID) > 10;
```